

Beispiel: Transaktionsprüfungen / CRUD

- **A=Atomicity:** Alle Operationen einer Transaktion werden entweder vollständig ausgeführt oder gar nicht.
- **C=Consistency:** Die Transaktion führt das System von einem konsistenten Zustand in einen anderen konsistenten Zustand.
- **I=Isolation:** Mehrere Transaktionen können gleichzeitig ausgeführt werden, ohne sich gegenseitig zu beeinflussen.
- **D=Durability:** Sobald eine Transaktion erfolgreich abgeschlossen ist, bleiben ihre Auswirkungen dauerhaft erhalten, selbst bei Systemausfällen.

Warum Transaktionen testen?

- **Datenintegrität:** Sicherstellen, dass die Daten in der Datenbank immer korrekt und konsistent sind.
- **Geschäftsregeln:** Verifizieren, dass alle businessrelevanten Regeln korrekt angewendet werden (z.B. keine negativen Lagerbestände).
- **Fehlerbehandlung:** Prüfen, ob das System bei Fehlern innerhalb einer Transaktion (z.B. fehlende Berechtigung, technisches Problem) korrekt ein Rollback durchführt.
- **Nebenläufigkeit (Concurrency):** Testen, wie das System reagiert, wenn mehrere Benutzer oder Prozesse gleichzeitig auf dieselben Daten zugreifen und Transaktionen ausführen.

Wie Transaktionen testen?

- **Positivtests (Erfolgreiche Transaktion):**

- **Ziel:** korrekt speichern (Commit).
- **Beispiel:** Geldüberweisung zwischen zwei Konten.
→ Check: beide Konten korrekt aktualisiert

- **Negativtests (Rollback):**

- **Ziel:** Rollback wenn Fehler innerhalb Transaktion
→ Check: Rollback vollständig
- **Beispiel:** Fehler während Überweisung (Unterbruch DB-Verbindung während Transaktion)
→ Check: Ausgangs- und Zielkonto korrekt

- **Methoden: Nebenläufigkeitstests (Concurrency Tests):**

- **Ziel:** Mehrere Transaktionen gleichzeitig ausführen, die sich gegenseitig beeinflussen könnten. Prüfen, ob Isolation gewahrt bleibt und keine Daten korrumpiert werden.
- **Beispiel:** Zwei Benutzer kaufen Produkt aber Bestand=1
→ Check: nur einer erfolgreich